

what is a program?

digitization and programming · session 2

agenda

- 1 last week
- 2 the instruction follower
- 3 four building blocks
- 4 it does what is written
- 5 working with the assistant

last week

who is
playing?



last week

last week, you were the receiver.

look at the card.

decide which colour it is.

write down the matching symbol.

wait for the next one.

that was an algorithm.
you were the machine.

steps so precise you could follow them
without understanding a thing.

last week

look at the card.

decide which colour it is.

write down the matching symbol.

wait for the next one.

one of these four will take you three weeks to hand over.

one more algorithm, live

- 1 stand in a row
- 2 look at your right neighbour
- 3 taller than them? swap places
- 4 end of the row? start again from the left
- 5 a full pass with no swap? you are sorted.

six volunteers. you are the machines: follow, do not think.

you just ran bubble sort.

computers sort exactly like this, just faster.

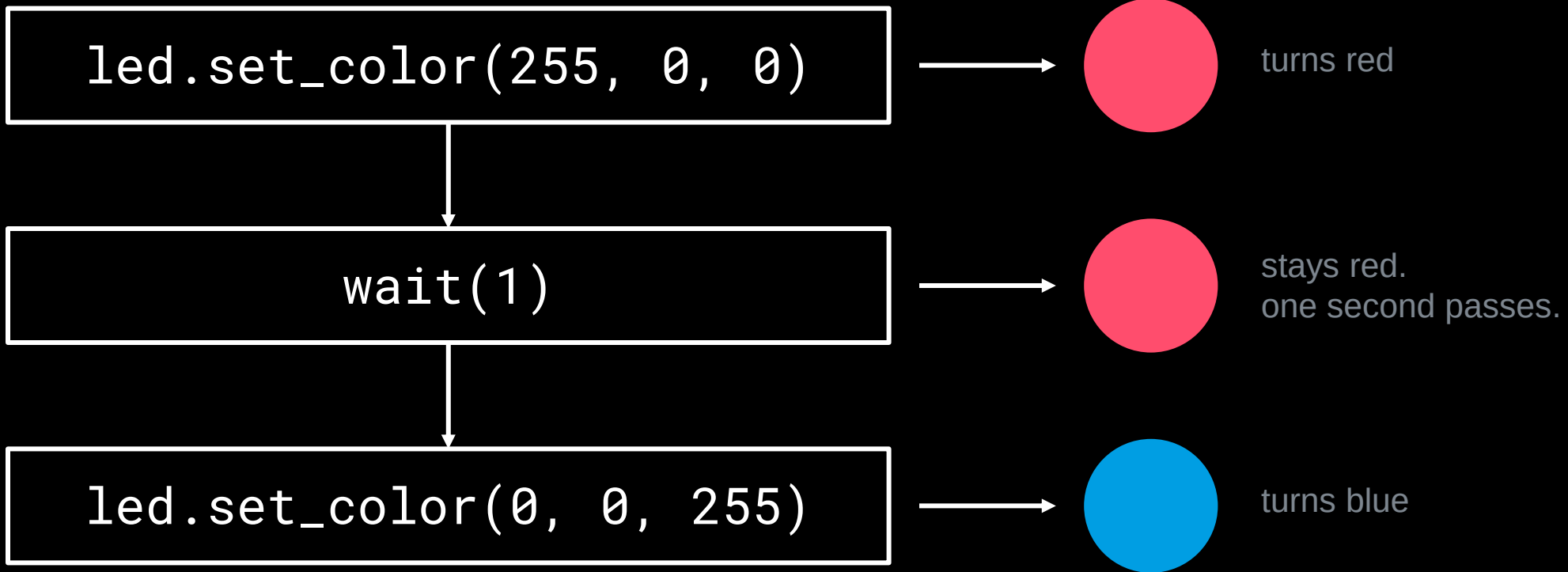
remember how many comparisons six people needed. cost returns later today.

a program is an algorithm
written for a machine.

complete. the machine adds nothing.

the instruction follower

a program is a list of instructions.
something follows that list.
exactly, and in order.



one line at a time, top to bottom. every line changes what you see.

one line

```
led.set_color(255, 0, 0)
```

one line

```
led.set_color(255, 0, 0)
```

led → which thing

set_color → what to do with it

(255, 0, 0) → with which values

one line

```
led.set_color(255, 0, 0)  
led.set_color(0, 0, 255)
```

same instruction. different values. different result.

four building blocks

1 one after another

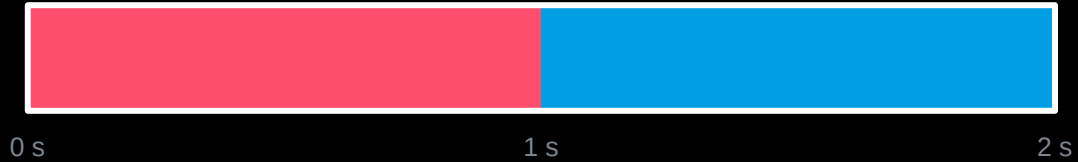
```
led.set_color(255, 0, 0)  
wait(1)  
led.set_color(0, 0, 255)
```

one line more or less is a different program.

with a pause

```
led.set_color(255, 0, 0)
wait(1)
led.set_color(0, 0, 255)
```

what the led shows



red for one second, then blue.

without a pause

```
led.set_color(255, 0, 0)
led.set_color(0, 0, 255)
```

what the led shows



red for a millisecond. you never see it.

pause in the middle

```
led.set_color(255, 0, 0)
wait(1)
led.set_color(0, 0, 255)
```

what the led shows



0 s 1 s 2 s

red for one second, then blue.

pause at the end

```
led.set_color(255, 0, 0)
led.set_color(0, 0, 255)
wait(1)
```

what the led shows



0 s 1 s 2 s

red for a millisecond. the machine
waits after everything happened.

same three lines, different order: a different program.

2 a value with a name

```
bright = 255  
led.set_color(bright, 0, 0)  
wait(1)  
led.set_color(bright, 0, 0)
```

one place to change instead of three.

3 again

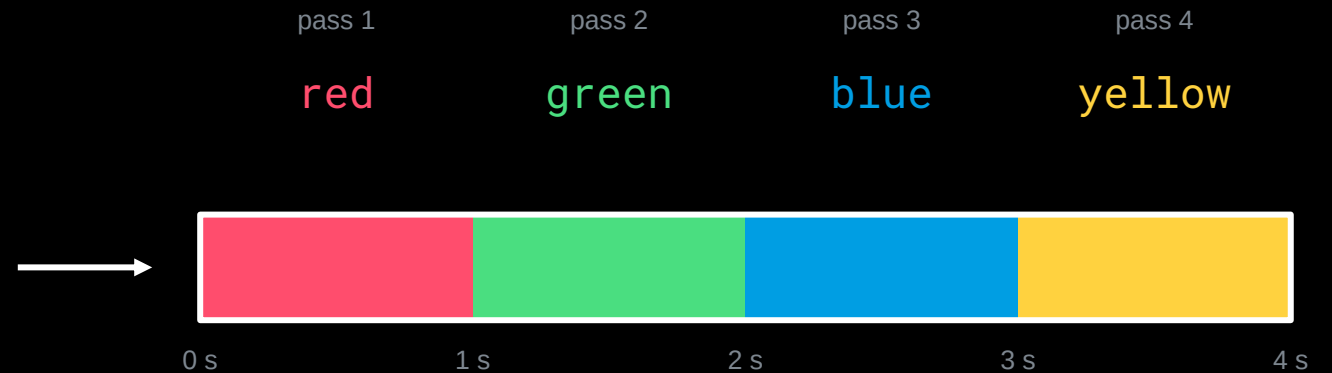
```
colours = [red, green, blue, yellow]
```

```
for colour in colours:  
    led.set_color(colour)  
    wait(1)
```

the loop you write

```
colours = [red, green,  
           blue, yellow]  
  
for colour in colours:  
    led.set_color(colour)  
    wait(1)
```

what actually happens



the same two lines, four times. one value per pass, one second per pass.

4 only if

```
for colour in colours:  
    led.set_color(colour)  
    if colour == yellow:  
        wait(3)  
    else:  
        wait(1)
```

now the run depends on something that changes along the way.

when does it end?

when does this end?

```
while True:  
    led.set_color(red)  
    wait(1)  
    led.set_color(blue)  
    wait(1)
```

never: the condition is always true, and the body has no way out.

fine for blinking forever. fatal for a receiver.

what does it cost?

both correct. not equal.

ask every number	99 questions
halve the range	7 questions

algorithms are compared by how their cost grows.
for your transfer: seconds per file.

it does what is written

an error message

```
traceback (most recent call last):  
  file "blink.py", line 7, in <module>  
    led.set_colour(255, 0, 0)  
attributeerror: no attribute 'set_colour'
```

line 2 → which file

line 3 → which line, and what stands there

line 4 → what actually went wrong

an error message is information, not a verdict.

read it out loud. most of the time it says exactly what is wrong.

now watch me get it wrong.

working with the assistant

two ways to ask

write me a program for the led.

this gets you code.

the led should be red for three seconds, then blue.
explain each line before you write it.

this gets you understanding.

never keep code you cannot change.

every hand-in ends with a live change. the oral exam does not ask who wrote it, it asks what it does.

today

- 1 one colour, on
- 2 four colours, in a loop
- 3 blinking, and you can change how fast

number 3 is the real one. you must be able to change the speed without hunting through the code.